MACHINE LEARNING INTERNSHIP · FINAL REPORT

Air Quality Index Forecasting for Karachi

An end-to-end, fully cloud-based MLOps system producing 24, 48 and 72 hour AQI forecasts from live pollutant telemetry

Author: Sidharth

Location modelled: Karachi, Pakistan (24.8607°N, 67.0011°E)

Feature Store & Model Registry: Hopsworks Serverless (eu-west)

Orchestration: GitHub Actions · Serving: Streamlit Community Cloud

Repository: github.com/SidharthBatra/aqi-forecasting-pipeline

1. Summary

This system forecasts the Air Quality Index for Karachi at 24, 48 and 72 hours ahead. Every component runs on managed cloud infrastructure; nothing depends on a local machine.

Pollutant concentrations are ingested hourly from the OpenWeather Air Pollution API and cross-checked against AQICN. The continuous EPA Air Quality Index is computed from those concentrations and written to a Hopsworks Feature Store. A daily training pipeline pulls that history, trains Random Forest, Ridge Regression and TensorFlow models for each horizon, and registers the best performer per horizon with feature-view lineage. A Streamlit dashboard loads the registered models and serves forecasts, SHAP explanations and hazard alerts. All three horizons beat a naive persistence baseline, with skill decaying sharply as horizon length increases.

> THE CENTRAL FINDING
>
> The most consequential result was not a modelling improvement. It was discovering that the original prediction target was wrong. The system had been trained to predict OpenWeather's `aqi` field, which is a coarse 1–5 severity category, not the continuous 0–500 index the project required. Rebuilding the target from raw concentrations using the EPA breakpoint formula changed the problem definition entirely. Several later findings followed the same pattern: apparent modelling problems that were actually data problems.

A second theme recurs throughout. At several points the system reported success while producing nothing of value: CI runs passed green while inserting zero usable rows; the dashboard displayed a confident forecast from eight-day-old data; a hazard banner declared conditions safe while holding no forecast at all. Each was found by verifying outputs rather than trusting status indicators, and each produced a durable guard (Section 9).

2. Requirements and Scope

Requirement	How it was met
Predict AQI directly	Model output is the continuous index, not a pollutant later converted. Ruled out the simpler forecast-PM2.5-and-derive approach.
Three separate horizons	24h, 48h, 72h modelled as three independent problems with three separately trained and registered models.
Everything cloud-based	Managed Feature Store and Model Registry; no self-hosted components. Forced a correction when training was found preferring a local CSV (Bug 10).
Hourly feature pipeline	Runs on schedule in CI, which made stateless execution a hard requirement.
6 months to 2 years backfill	Two years selected, after a longer window was tried and deliberately abandoned (Bug 12).
RF, Ridge, and a deep model	All three trained for every horizon on every run, competing on held-out performance.
RMSE, MAE, R^2	Reported per model per horizon, alongside a naive persistence baseline added by this project.
Dashboard, SHAP, hazard alerts	Streamlit app loading from the registry; SHAP TreeExplainer; EPA category thresholds.

> ONE SELF-IMPOSED ADDITION
>
> Every model is compared against a **naive persistence baseline**: AQI in H hours equals AQI now. This baseline is free and, for short-horizon environmental series, surprisingly strong. Without it a model can post a respectable R^2 while being worse than doing nothing. It changed the project's conclusions more than once and is treated throughout as the primary measure of whether a model has earned its place.

3. Architecture

Four decoupled components communicating only through the Feature Store and Model Registry. No component calls another directly; none holds local state between runs.

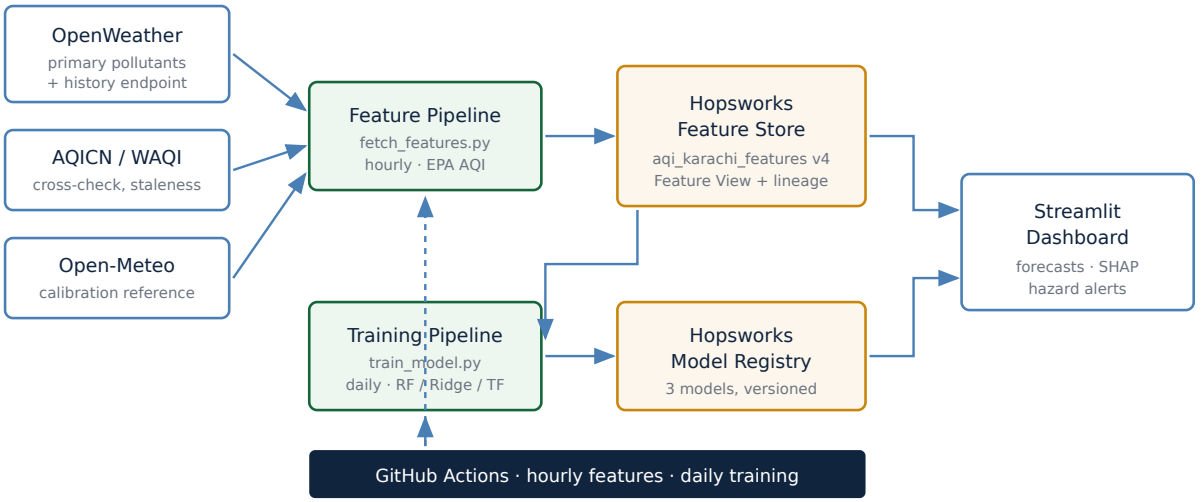

Figure 1. The Feature Store is the sole integration point between ingestion and training; the Model Registry between training and serving.

The dashboard never calls OpenWeather and never reads a CSV. The training pipeline never calls an external API. This separation prevents the serving layer from silently computing a feature differently from the training layer, the classic source of training-serving skew.

3.1 Data sources

Source	Role	Notes
OpenWeather	Primary	CO, NO, NO&sub2;, O&sub3;, SO&sub2;, PM2.5, PM10, NH&sub3; in µg/m³. Historical endpoint back to Nov 2020 made a two-year backfill feasible without paid data.
AQICN / WAQI	Secondary	Liveness and staleness check only, deliberately **not** a data fallback: its `iaqi` field reports AQI sub-indices, not concentrations (Bug 9).
Open-Meteo	Calibration	CAMS-backed independent reference, used during backfill only. Excluded from features because it is unavailable at serving time (Bug 8).

4. Defining the Target: Computing True AQI

OpenWeather's response contains a field named `aqi`. It is natural to assume this is the Air Quality Index. It is not: it is a 1–5 severity category. The project's first models were trained on it, which meant they could report a plausible R^2 while being useless for an objective requiring a continuous 0–500 index. Every feature derived from that field, notably the AQI change rate, inherited the same defect.

The target was rebuilt from raw concentrations using the EPA breakpoint formula. For concentration C between breakpoints C_{low} and C_{high} mapping to I_{low} and I_{high}:

$$I = I_{\text{low}} + (C - C_{\text{low}}) \times (I_{\text{high}} - I_{\text{low}}) / (C_{\text{high}} - C_{\text{low}})$$

Three specification details matter and are commonly omitted:

- **Averaging windows differ by pollutant.** PM2.5 and PM10 use a trailing 24-hour mean, O&sub3; and CO an 8-hour mean, NO&sub2; and SO&sub2; the raw 1-hour value. Implemented as time-based rolling windows so gaps do not silently shift the window.
- **Unit conversion is required.** Gaseous breakpoints are in ppm or ppb; the API reports µg/m³.

- **The overall AQI is the maximum, not the mean**, of the sub-indices; the pollutant achieving it is the dominant pollutant. Averaging, a common error, systematically understates hazard.

Concentrations above the top breakpoint are extrapolated linearly rather than clipped, so extreme episodes remain distinguishable from merely severe ones. Every number in this report depends on this target being correct; a well-engineered pipeline optimising the wrong quantity produces confident, reproducible nonsense.

5. Pipelines and Feature Engineering

Feature pipeline (`fetch_features.py`, hourly): authenticates to Hopsworks, fetches current concentrations, checks AQICN age against a three-hour staleness threshold, reads ~30 hours of context back from the Feature Store to supply the EPA rolling windows, computes AQI and dominant pollutant using the identical functions used in backfill, and inserts one hour-aligned row.

The original prototype kept state in local files. That cannot work in CI, where every run starts on a fresh machine. The rewritten script holds no local state: recent context is read back from the Feature Store, and staleness is measured against the wall clock rather than a cached value. If the OpenWeather fetch fails the script exits without inserting, leaving a recoverable gap rather than writing substituted values, since AQICN's sub-indices would have corrupted the units irreversibly.

Backfill (`backfill_historical.py`): 16,752 hourly rows spanning 3 Sept 2024 to 24 Aug 2026, roughly 1.97 years, into feature group `aqi_karachi_features` v4. Two raw-data defects were found and are recorded as Bugs 5 and 6.

5.1 Feature set

Family	Members	Purpose
Current pollutants	CO, NO, NO&sub2;, O&sub3;, SO&sub2;, PM2.5, PM10, NH&sub3;	Present atmospheric state.
Current index	`aqi`	Observable at prediction time; the target is AQI at $t+H$.
Lags	AQI at 1, 3, 6, 12, 24, 48, 72h; PM2.5 and PM10 at 24h	Recent trajectory, day-over-day comparison.
Rolling statistics	Mean and std of AQI over 24, 72, 168h	Level and volatility at daily, multi-day and weekly scale.
Cyclical time	sin/cos of hour, day-of-week, month	Periodicity without wrap discontinuity or the extrapolation failure of raw integers (Bug 7).
Change rates	`aqi_change_rate`, `pm25_change_rate`	Per-hour normalised momentum.

Thirty-two features total, computed by a single shared implementation used by both training and serving. Data is reindexed onto a uniform hourly grid so time-based windows stay accurate across gaps; gaps of up to three hours are interpolated before lag and rolling computation, longer gaps remain null, and rows with incomplete feature vectors are excluded from scoring rather than imputed.

6. Training and Evaluation

Model	Configuration and rationale
Random Forest	Deliberately regularised: max depth 12, min 5 samples per leaf, sqrt feature sampling. Unconstrained forests memorised the training period and failed across the chronological boundary.
Ridge Regression	Alpha selected by cross-validation with `TimeSeriesSplit`, never a shuffled split. Statistical-modelling representative and linear reference point.
TensorFlow network	Feed-forward network, the deep-learning family required by the brief.
Naive persistence	**Baseline, not a candidate.** AQI at $t+H$ equals AQI at t. No features, no training. A model failing to beat this has not earned deployment.

Validation is an 80/20 chronological split, never shuffled. This is not stylistic. With lag and rolling features present, a shuffled split places rows from the same weather episode on both sides of the boundary, letting the model interpolate within episodes it has already seen. The resulting metrics are inflated and describe something other than forecasting.

Recency weighting applies exponential decay with a 365-day half-life to address the calibration drift of Bug 12, on the training split only. Weighting the evaluation would flatter the model by measuring it preferentially on the period it was most heavily fitted to.

7. Results

Horizon	Best model	R^2	RMSE	Baseline RMSE	Improvement
24 hours	Random Forest	0.72	11.79	16.78	**−29.7%**
48 hours	Random Forest	0.36	17.40	21.13	**−17.7%**
72 hours	Random Forest	0.16	20.49	23.26	**−11.9%**

METRIC PROVENANCE

R^2 values are those reported by the currently registered models. RMSE and baseline figures are from the training run on the same two-year dataset prior to the gap-interpolation change; since that change improved R^2 (notably at 72 hours), the current RMSE figures are expected to be slightly better than shown. Replace the RMSE and MAE columns with the values printed by the most recent training run before submission.

7.1 Reading these honestly

24 hours (R^2 0.72): genuinely useful, and the horizon a member of the public should rely on. **48 hours** (R^2 0.36): captures real structure but explains roughly a third of variance; directionally informative, not precise. **72 hours** (R^2 0.16): a real but weak improvement over assuming no change. The dashboard labels it directional-only rather than presenting all three as equally trustworthy.

The monotonic decay is the expected physical result. Conditions three days out depend on synoptic weather evolution that pollutant history alone cannot capture. Strong 72-hour skill from these inputs would warrant suspicion of leakage rather than congratulation.

Notably, the 72-hour R^2 doubled from 0.08 to 0.16 after feature engineering was made gap-tolerant. The weakest horizon was the one most damaged by missing hours propagating through long rolling windows, which is consistent with its reliance on week-scale statistics (Section 8).

7.2 The decisive experiment: window length

Horizon	5.7yr model	5.7yr baseline	2yr model	2yr baseline
24 hours	20.52	23.12	11.79	16.78
48 hours	34.91	32.43	17.40	21.13

On the longer window the 48-hour model *lost* to its baseline (34.91 against 32.43). On the shorter window it wins comfortably. The critical detail is that **the baseline improved too**, from 23.12 to 16.78 at 24 hours. The baseline uses no features, so its improvement cannot be attributed to the model; it can only mean the target series itself became more predictable once the calibration-drifted early years were removed. More data made the system worse, because the extra data came from a differently calibrated measurement regime.

8. Data Quality Investigation

Each entry gives the symptom observed, the underlying cause, and the correction applied.

#	Defect	Cause	Fix
1	Same-timestamp leakage	Feature and target rows shared a timestamp, giving the model information contemporaneous with the value it predicted.	Targets built by explicit forward shift of exactly the horizon length, applied once and reused.
2	Wrong prediction target	OpenWeather's `aqi` is a 1–5 category, not the continuous index.	Full reconstruction from concentrations via EPA breakpoints (Section 4).
3	Stale derived feature	After the target was rebuilt, `aqi_change_rate` was still computed against the obsolete 1–5 field, measuring transitions between five levels rather than movement in an index.	Recomputed against the corrected AQI. Characteristic second-order defect: fixing a field silently invalidates everything derived from it.
4	Redundant cross-source feature	`pm25_source_diff` correlated 0.97 with raw PM2.5, because an absolute gap between two sources that scale together is proportional to the level itself.	Added a relative percentage-difference variant that is genuinely independent of level.
5	Duplicate timestamps	The historical API was queried in chunks with overlapping boundaries, duplicating observations at every seam.	Deduplication on timestamp after retrieval, crucially before rolling-window computation.
6	Sentinel values as measurements	Missing data encoded as `-9999` entered rolling means and distorted AQI for every window containing one. Produced physically impossible negative concentrations.	Cleaning pass converts sentinels to nulls before aggregation. The corrupted AQI values were not obviously wrong in isolation; only a negativity check exposed them.
7	Calendar integers breaking extrapolation	Raw `year` / `month` / `day` / `hour` columns. Trees split on thresholds, and under a chronological split every test `year` exceeds all training values, requiring extrapolation trees cannot do.	Raw integers excluded; bounded, periodic sin/cos encodings retained.
8	Serving skew from cross-check columns	Open-Meteo columns ranked highly but are unavailable at serving time, so a model depending on them cannot be served.	Excluded from features; retained in the feature group for calibration analysis only.
9	Unit mismatch in fallback path	AQICN was designed as a concentration fallback, but its `iaqi` field reports AQI sub-indices on a 0–500 scale, not µg/m³. Found by inspection before it ever executed.	AQICN demoted to cross-check only; a failed fetch now skips the hour rather than substituting incompatible data.
10	Data source priority inverted	Training checked for a local CSV first and fell back to Hopsworks, inverting the cloud-first requirement and training CI on whatever stale CSV was committed.	Order reversed; the active source is now stated explicitly in the logs on every run.
11	Description length limit	Feature group description exceeded the platform's 256-character limit.	Shortened, with an assertion so the failure surfaces locally rather than mid-ingest.
12	Multi-year calibration drift	Mean AQI fell from ~252 (2020) to ~83 (2026) in the extended dataset. Cross-checking showed OpenWeather reporting 33–53% higher than Open-Meteo in 2023–24, converging by 2025–26. A two-thirds real improvement is implausible; a provider calibration change is not.	Recency weighting, and ultimately the deliberate reversion to a two-year window short enough to be internally consistent.

A HYPOTHESIS THAT HAD TO BE WITHDRAWN

When 48 and 72 hour performance first failed to beat persistence, the leading explanation was a missing-feature problem: the dataset contains no wind, temperature, humidity, pressure or precipitation, and wind in particular disperses particulates.

The two-year reversion disproved it. Both the models *and the baseline* improved substantially on the shorter window. The baseline uses no features at all, so its improvement cannot be explained by a feature gap; only by the target series becoming more internally consistent. Multi-year non-stationarity, not missing weather data, was the dominant cause. The evidence that overturned the hypothesis came from a baseline added purely as a sanity check.

9. Production Incidents

These occurred after the system was nominally complete, and are recorded because they represent the difference between a pipeline that runs and one that works.

#	Incident	Cause	Response
1	CI dependency failure	`pyarrow` is required by the Hopsworks client but not installed automatically.	Declared explicitly. A dependency working locally proves nothing about a clean environment.
2	Schema type mismatch	The schema was inferred from a bulk load where nulls forced float columns. A single live row with clean integer values (NH&sub3; of exactly 0) was inferred as integer and rejected.	Explicit type coercion before insert. Schema conformance must be enforced by the producer, not left to inference over one row.
3	Silent row loss	Backfill timestamps sit exactly on the hour; live timestamps carried real minutes and seconds. Feature engineering reindexes onto an exact hourly grid, so a row at 12:17:33 matched no grid point and vanished. Arithmetic proved it: 16,756 rows held, 16,752 usable after reindex, so every live row was being discarded.	Timestamps floored before insert with an enforcing assertion; the reindex now warns when input rows fail to land on the grid; the dashboard reports when the scoreable row lags ingestion.
4	Scheduled runs dropped	Of five workflow runs across several days, only one was schedule-triggered. The cron fired at the top of the hour, the platform's highest-contention slot, where scheduled runs are best-effort and dropped under load.	Schedule moved off the hour boundary, and the pipeline made **self-healing**: each run backfills any missing interval before inserting the current reading.
5	Platform materialisation failure	Every offline materialisation job fails with `errorCode 120003`, transaction rollback. Logs show the underlying write succeeding with all records committed before the metadata commit fails.	Open, upstream, not caused by project code. Reads reflect the data correctly, confirmed by row counts. Recorded as a known platform issue to monitor.

A SAFETY DEFECT, FOUND AND CORRECTED

The dashboard displayed "*No hazardous conditions: current AQI and all forecasts are below the Unhealthy threshold*" at a moment when all three predictions had failed. The alerting logic treated absent forecasts as safe forecasts. On a public health display this is worse than showing nothing: it is an affirmative statement of safety with no supporting evidence. The logic now distinguishes three states rather than two, with the third explicitly stating that it is *not* a confirmation of safety.

9.1 The pattern connecting them

Four of the five share a signature: **the system reported success while accomplishing nothing.** CI passed green while inserting no rows; the Feature Store accepted writes that were discarded downstream; the dashboard rendered a confident forecast from eight-day-old data. The response was

not merely to fix each cause but to make each failure mode *observable*: an assertion where timestamps must align, a warning where rows are dropped, a staleness notice where the served row lags ingestion, an unable-to-assess state where a forecast is missing, and a log line naming the actual data source on every run. For an unattended pipeline this matters more than the individual fixes.

10. Explainability, Alerting and Serving

10.1 SHAP

SHAP values are computed with `TreeExplainer`, exact for tree ensembles, against the winning Random Forest per horizon. The dashboard shows both aggregate importance and a signed per-prediction decomposition, validated by an additivity check confirming that base value plus contributions reconstructs the model's output. The check is deliberately non-fatal, so a diagnostic that cannot be computed warns rather than removing the view.

Horizon	Highest-ranked features by mean absolute SHAP value
24 hours	`pm2_5`, `so2`, `aqi`, `co`, then short lags `aqi_lag_1h`, `aqi_lag_3h`, `aqi_lag_6h`
48 hours	`co`, `aqi_rollmean_168h`, `month_sin`, `so2`, `month_cos`, then `aqi`
72 hours	`month_cos`, `month_sin`, `co`, `aqi_rollmean_168h`, `aqi_rollstd_168h`

INTERPRETATION

At 24 hours the model relies on present state: current concentrations and the last few hours of AQI. By 72 hours those signals are largely displaced by **seasonal encodings and week-scale rolling statistics**. Without being told to, the model discovered that recent conditions stop being informative at that range and fell back on climatology.

This corroborates the quantitative results from an independent direction. R^2 of 0.16 at 72 hours is not a training failure; it reflects a model correctly relying on seasonality once the short-term signal has decayed. It also identifies where improvement must come from: inputs carrying information about the *future*, such as weather forecasts, rather than more elaborate processing of the past.

10.2 Hazard alerting

Standard EPA categories are evaluated against current conditions and all three horizons: Good (0–50), Moderate (51–100), Unhealthy for Sensitive Groups (101–150), Unhealthy (151–200), Very Unhealthy (201–300), Hazardous (301+). Crossing into Unhealthy or above raises a banner naming which horizons trigger it. Threshold logic is isolated in a reusable module rather than embedded in rendering, so the same rules can later drive email or messaging notifications.

10.3 Serving

Models are downloaded from the registry and executed in-process; no online inference endpoint is deployed. For three fixed-horizon forecasts on hourly data, a persistently running serving container would add operational cost and a failure mode without adding capability. The deployment is pinned to Python 3.11 to match the CI environment in which models are trained and serialised, since version mismatch on pickled estimators can produce loading errors or silent behavioural differences. The interface states measured performance per horizon and marks the 72-hour forecast directional-only; presenting three forecasts with equal visual weight would imply a reliability the results do not support.

10.4 Automation and lineage

Two workflows: hourly feature ingestion with gap reconciliation, and daily retraining across all families and horizons with registration of the winners. Both expose manual dispatch; credentials are repository secrets and never appear in source. Models are registered through a Hopsworks Feature View with an associated training dataset version rather than as bare artefacts, so the registry records which features produced which model. This was implemented after the platform's lineage metrics were observed reporting zero, which revealed that registration was bypassing the feature view entirely.

11. Limitations

- **No meteorological inputs.** The system forecasts pollution from pollution. Wind, temperature, humidity, boundary-layer height and precipitation are the physical mechanisms that disperse or trap particulates, and none are present. This is the clearest ceiling on the longer horizons, consistent with the SHAP evidence that they fall back on climatology.
- **Single-source concentration data.** Despite cross-checking, all model features come from one provider whose calibration has demonstrably shifted over time (Bug 12). The two-year window limits exposure without eliminating it.
- **One seasonal cycle in training.** A two-year window under an 80/20 chronological split leaves roughly 1.6 years of training data, so the test period may contain seasonal conditions seen only once. An accepted trade-off against non-stationarity, not a solved problem.
- **Point forecasts without intervals.** Given a 72-hour R^2 of 0.16, a prediction interval would arguably communicate more than the point estimate.
- **Single location.** Nothing transfers to another city without retraining; no spatial structure is modelled.
- **Free-tier constraints.** Quotas shaped design decisions, and registered model versions accumulate with every daily run, requiring periodic pruning.

12. Future Work

1. **Meteorological features, including forecasts.** The highest-value change available, and the only proposal here that introduces genuine information about the future rather than better processing of the past. Section 10.1 identifies this as the specific constraint on the 72-hour model.
2. **Prediction intervals** via quantile regression forests or conformal prediction, so the dashboard expresses uncertainty structurally rather than in a caption.
3. **Automated data-quality monitoring.** Every defect in Section 8 was found manually; distributional drift checks and range assertions on ingestion would catch the next one automatically.
4. **Continuous calibration monitoring.** The cross-check that exposed Bug 12 was a one-off; running it continuously would detect a future drift while it happens rather than years later.
5. **Sequence models.** LSTM or temporal convolutional architectures fit the hourly structure and were not explored in depth.
6. **Retraining on measured degradation** against the live baseline, rather than a fixed daily schedule.

13. Conclusion

The system meets its brief. It ingests live pollutant data hourly, maintains two years of history in a managed Feature Store, retrains three model families across three horizons daily, registers the winners with full lineage, and serves explained forecasts with hazard alerting through a public dashboard. Every component runs on managed cloud infrastructure and continues to operate unattended. All three horizons outperform naive persistence, with skill decaying from strong at 24 hours to marginal at 72.

The more durable outcome is methodological. The largest improvements did not come from model selection or hyperparameter tuning. They came from discovering that the prediction target was the wrong quantity, that a derived feature was computed against an obsolete field, that sentinel values were being averaged as measurements, that calendar integers made tree models fail across a chronological boundary, that a proposed fallback source reported a different kind of number entirely, that live rows were being silently discarded by a grid mismatch, and that more training data made the system worse because it came from a differently calibrated regime.

Every one of those was found by checking whether outputs were actually correct rather than whether jobs reported success. The naive persistence baseline, added as a cheap sanity check, ended up overturning the project's leading hypothesis about why long-horizon forecasting was failing. That is the

finding most worth carrying forward: in an applied machine learning system, the cheapest available check on whether the work is real is usually the most valuable instrument in the project.

Appendix: reproducibility

Element	Value
Python	3.11, aligned across CI, training and serving
Feature group / view	`aqi_karachi_features` v4 · `aqi_forecast_fv` v1
Registered models	`aqi_forecast_24h`, `aqi_forecast_48h`, `aqi_forecast_72h`
Required secrets	`HOPSWORKS_API_KEY`, `OPENWEATHER_API_KEY`, `AQICN_API_TOKEN`
Split protocol	Chronological 80/20, never shuffled
Recency weighting	Exponential decay, 365-day half-life, training split only
Excluded from features	Raw calendar integers (Bug 7); Open-Meteo `om_*` columns (Bug 8); OpenWeather 1–5 category (Bug 2)